L01 – Power Flow Fundamentals

Foundations Lecture Series

Course Instructor

Microgrid 3-phase Security AI Project

Foundations Lecture 1

Keywords: bus, branch, per-unit, Y-bus, Newton-Raphson, slack/PV/PQ

Lecture roadmap

1. Why power flow? What problem are we actually solving?
2. The setup: buses, branches, per-unit, bus types
3. The math: Y-bus and the power-flow equations
4. Solving it: Newton-Raphson in one slide
5. A worked three-bus example
6. pandapower preview: how this looks in code
7. Limits and what comes in L02–L04

What you will leave with

The ability to write down the power-flow equations from a one-line diagram, classify each bus correctly, and run `pp.runpp(net)` understanding what every line of input and output means.

The grid as a giant electrical circuit

- Sources (generators, PV, batteries) and sinks (loads) connected by transmission and distribution lines.
- Continuous instantaneous balance: $\sum P_{\text{gen}} = \sum P_{\text{load}} + P_{\text{losses}}$.
- Operators care about voltages, currents, and limits in *steady state*.
- *Power flow* = solve for the steady-state operating point given the loads and generator setpoints.

What it is not

Not dynamics (no rotor angle ODEs). Not protection (no fault current calculation). Not market clearing (no economic dispatch). *Just the snapshot.*

What operators actually need to know

- Voltage at every bus – magnitude and angle.
- Active and reactive power flow on every line and transformer.
- Are all voltages between 0.95 and 1.05 pu?
- Is any line loaded above 100% of its current rating?
- How much P and Q is the swing generator actually picking up?

Use cases

- Dispatch / operations planning
- Contingency analysis (this course)
- Network expansion planning
- Market clearing (security-constrained OPF)
- Hosting capacity studies for DER

Power flow in one slide

Given

- Network topology (which bus connects to which through what)
- Line / transformer impedances
- Generator and load setpoints

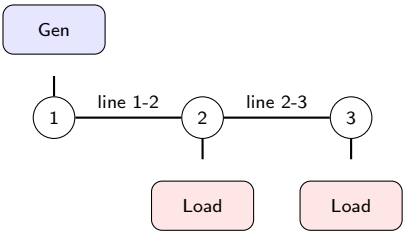
Find

- Voltage magnitude $|V_i|$ and angle θ_i at every bus

Then derive

- Currents and active/reactive power on every branch
- Total losses
- Generator outputs at the slack bus

Buses, branches, generators, loads



- **Bus** – a node. Where things connect.
- **Branch** – a line, transformer, or switch connecting two buses.
- **Generator** – power source. Injects P_g, Q_g into a bus.
- **Load** – power sink. Draws P_d, Q_d from a bus.
- **State variables** at each bus: $|V_i|, \theta_i$.

Per-unit system: why and how

Why

- A transformer changes voltage levels. Without per-unit, every quantity has different units on each side. Confusing.
- In per-unit, the same line impedance has the same value on both sides of a transformer.
- Per-unit voltage is approximately 1.0 everywhere when the system is healthy.

How

- Pick S_{base} (e.g., 100 MVA) and V_{base} per voltage zone.
- Compute $Z_{\text{base}} = V_{\text{base}}^2 / S_{\text{base}}$, $I_{\text{base}} = S_{\text{base}} / (\sqrt{3}\, V_{\text{base}})$.
- Divide every quantity by its base: $V_{pu} = V / V_{\text{base}}$, etc.

Rule of thumb

If you see "$V_i = 0.96$ pu", that means 96% of nominal voltage — usually borderline-low.

Bus types: slack, PV, PQ

Type	Known	Unknown	Physical interpretation	Number per system
Slack	$\lvert V\rvert, \theta$	P, Q	Reference + absorbs imbalance	exactly 1
PV	$P, \lvert V\rvert$	θ, Q	Generator with voltage control	several
PQ	P, Q	$\lvert V\rvert, \theta$	Load or fixed-output source	many

Why this taxonomy

Each bus contributes two unknowns ($|V_i|, \theta_i$) and two equations (P_i, Q_i).

- At a PQ bus we know P and Q from the load, so we solve for $|V|$ and θ.
- At a PV bus the generator controls $|V|$; Q is whatever the generator must supply.
- The slack bus fixes θ as the angle reference and picks up losses.

Bus admittance matrix Y_{bus}

For each line between bus i and bus j with series admittance $y_{ij} = 1/(R + jX)$:

$$Y_{ii} = \sum_{j \neq i} y_{ij} + y_{i,\text{shunt}}$$
$$Y_{ij} = -y_{ij}, \quad j \neq i$$

Stack into a complex $N \times N$ matrix:

$$Y_{\text{bus}} = G + jB \quad \in \mathbb{C}^{N \times N}$$

Properties

- Symmetric: $Y_{ij} = Y_{ji}$.
- Sparse for real networks (each bus connects to a few neighbours).
- Pre-computed once at the start of a power-flow solve.

The power-flow equations

The current injected at bus i is $I_i = \sum_j Y_{ij} V_j$. The complex power injection is

$$S_i = P_i + jQ_i = V_i\, I_i^* = V_i \sum_j Y_{ij}^*\, V_j^*.$$

Writing $V_i = |V_i|\angle\theta_i$ and $Y_{ij} = |Y_{ij}|\angle\alpha_{ij}$ and expanding:

$$P_i = \sum_j |V_i||V_j||Y_{ij}| \cos(\theta_i - \theta_j - \alpha_{ij})$$

$$Q_i = \sum_j |V_i||V_j||Y_{ij}| \sin(\theta_i - \theta_j - \alpha_{ij})$$

The key observation

These equations are *nonlinear* in $|V|$ and θ — products of voltage terms and trig functions. No closed-form solution. We must iterate.

Why nonlinearity matters

- Linear systems: one unique solution, closed-form.
- Nonlinear systems: potentially multiple solutions, no closed form, iterative solver required.
- Power-flow equations can have multiple mathematical solutions; we want the operationally relevant one.
- Solver may not converge. *Always* check.

Practical consequences

- Initial guess matters. Flat start: $|V| = 1, \theta = 0$.
- Heavily loaded systems are harder to solve.
- Long radial feeders are notorious.
- Tools provide `net.converged` – treat it as a load-bearing check.

Newton-Raphson in one slide

We want to solve $F(x) = 0$ where F is the vector of power mismatches and $x = [\theta; |V|]$ for non-slack buses. **Update rule:**

$$x^{(k+1)} = x^{(k)} - J(x^{(k)})^{-1} F(x^{(k)})$$

Where:

- $F^{(k)}$ = mismatch vector $[\Delta P; \Delta Q]$
- $J^{(k)}$ = Jacobian, partial derivatives of F w.r.t. x

Iterate until $\|F^{(k)}\|_\infty < \text{tol}$ (e.g. 10^{-8}).

Typical convergence

Well-posed problems converge in 3–5 iterations. Each iteration costs O(sparse linear solve), very fast in practice.

The Jacobian

Block structure for n non-slack buses:

$$J = \begin{bmatrix} \dfrac{\partial P}{\partial \theta} & \dfrac{\partial P}{\partial |V|} \\ \dfrac{\partial Q}{\partial \theta} & \dfrac{\partial Q}{\partial |V|} \end{bmatrix}$$

- Off-diagonal entries non-zero only when the two buses share a branch.
- Sparse with the same pattern as Y_{bus}.
- Re-evaluated at each NR iteration.
- Sparse LU or iterative solver handles $J\Delta x = -F$ efficiently.

Variants you will see

"Decoupled PF" and "fast-decoupled PF" approximate J by zeroing off-diagonal blocks. Faster per iteration, slightly more iterations. Default in many tools.

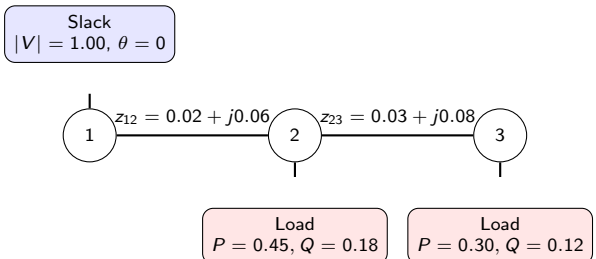
What can go wrong

1. △ **Non-convergence.** The solver hits the iteration limit. Usually the system is heavily loaded, the topology is bad, or the load exceeds the line capacity.
2. △ **Multiple solutions.** NR finds one. If the initial guess is far from the operating point, you might land on the wrong root.
3. △ **Singular Jacobian.** Happens near voltage collapse (the "nose" of the PV curve).
4. △ **Bad input data.** Mistyped impedance, missing slack, isolated bus, wrong base voltage.

Defensive habit

Always inspect `net.converged`, then sanity-check voltages and loadings before trusting any downstream analysis.

Three-bus example: setup



- All quantities are in per-unit on a 100 MVA base.
- Bus 1: slack ($|V| = 1$, $\theta = 0$).
- Buses 2, 3: PQ buses.
- Goal: find $|V_2|, \theta_2, |V_3|, \theta_3$.

Three-bus example: build Y_{bus}

Line admittances:

$$y_{12} = 1/(0.02 + j0.06) = 5.00 - j15.00$$
$$y_{23} = 1/(0.03 + j0.08) = 4.10 - j10.93$$

Y_{bus} matrix:

$$Y_{\text{bus}} = \begin{bmatrix} y_{12} & -y_{12} & 0 \\ -y_{12} & y_{12} + y_{23} & -y_{23} \\ 0 & -y_{23} & y_{23} \end{bmatrix}$$

Each off-diagonal entry is $-y_{ij}$; each diagonal sums the admittances of branches touching that bus. The matrix is symmetric and (apart from the row/column corresponding to the slack) used directly in NR.

Three-bus example: NR sketch

1. **Initialize** (flat start): $|V_2| = |V_3| = 1$, $\theta_2 = \theta_3 = 0$.
2. **Compute mismatch** $\Delta P_2, \Delta Q_2, \Delta P_3, \Delta Q_3$ from the PF equations.
3. **Compute Jacobian** $J^{(0)}$ at the current $(|V|, \theta)$.
4. **Solve** $J\,\Delta x = -F$ for $\Delta x = [\Delta\theta_2, \Delta\theta_3, \Delta|V_2|, \Delta|V_3|]$.
5. **Update** $x \leftarrow x + \Delta x$.
6. **Repeat** steps 2–5 until $\|F\|_\infty < 10^{-8}$.

Numerical result (converged)

$|V_2| \approx 0.964, \theta_2 \approx -2.3^\circ, |V_3| \approx 0.944, \theta_3 \approx -3.6^\circ$. Converges in 3 iterations. You can reproduce this with `pp.runpp(net)` on a 100 MVA, 20 kV system.

Minimal pandapower workflow

```
import pandapower as pp

net = pp.create_empty_network()
b1 = pp.create_bus(net, vn_kv=20.0, name="slack")
b2 = pp.create_bus(net, vn_kv=20.0, name="load1")
b3 = pp.create_bus(net, vn_kv=20.0, name="load2")

pp.create_ext_grid(net, bus=b1, vm_pu=1.0)
pp.create_line_from_parameters(
    net, b1, b2, length_km=1.0,
    r_ohm_per_km=0.08, x_ohm_per_km=0.24,
    c_nf_per_km=0.0, max_i_ka=0.4)
pp.create_line_from_parameters(
    net, b2, b3, length_km=1.0,
    r_ohm_per_km=0.12, x_ohm_per_km=0.32,
    c_nf_per_km=0.0, max_i_ka=0.4)

pp.create_load(net, bus=b2, p_mw=1.5, q_mvar=0.6)
pp.create_load(net, bus=b3, p_mw=1.0, q_mvar=0.4)

pp.runpp(net)
print(net.res_bus)
print(net.res_line)
```

Reading pandapower results

`net.res_bus` – one row per bus:

- `vm_pu` – voltage magnitude in per-unit
- `va_degree` – voltage angle in degrees
- `p_mw`, `q_mvar` – net injection at the bus

`net.res_line` – one row per line:

- `loading_percent` – $|I_{\max(\text{from,to})}| / I_{\text{rated}} \times 100$
- `i_from_ka`, `i_to_ka`, `pl_mw`, `ql_mvar`

Always do these three checks

1. `net.converged` is True.
2. No `vm_pu` outside $[0.95, 1.05]$ (unless expected).
3. No `loading_percent` above 100 (unless expected).

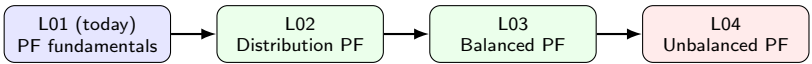
What this lecture assumed

1. △ **Balanced three-phase.** We treated each bus as a single complex voltage. This implicitly assumes the three phases are identical up to a 120° rotation.
2. △ **Steady state.** No dynamics, no transient stability, no protection.
3. △ **Fixed topology.** No outages yet.
4. △ **AC, sinusoidal.** No DC links, no harmonics.

When the balanced assumption is OK

At transmission voltage levels with three-phase generation and roughly symmetric loads, the assumption is usually fine. We will see in L02–L04 that distribution networks (and especially LV microgrids) are a very different story.

Where we go from here



- **L02** – radial feeders, R/X ratio, BFS, DER on distribution.
- **L03** – when the balanced assumption is appropriate; positive sequence equivalents.
- **L04** – symmetrical components, three-phase PF, VUF, why microgrids need it.

Homework

Required

1. By hand, write down Y_{bus} for the three-bus system in this lecture.
2. By hand, compute the power mismatch at the flat start.
3. Reproduce the three-bus system in pandapower and run `pp.runpp(net)`. Confirm `net.converged` is True.
4. Compare `net.res_bus.vm_pu` to the typical values quoted on the worked-example slide.

Optional

1. Double the line impedances and re-run. How does $|V|$ change at bus 3?
2. Add a PV-type generator at bus 3 and observe how θ changes.
3. Try `pp.runpp(net, algorithm="bfsw")` and compare voltages to NR. (We will come back to BFSW in L02.)

Exit checklist

1. ✓ I can explain what "solving the power flow" means in one sentence.
2. ✓ Given a one-line diagram, I can classify each bus as slack, PV, or PQ.
3. ✓ I can write down Y_{bus} from line impedances.
4. ✓ I can state the power-flow equations and explain why they are nonlinear.
5. ✓ I know what Newton-Raphson does and what convergence means.
6. ✓ I can run `pp.runpp(net)` and read `net.res_bus.vm_pu` and `net.res_line.loading_percent`.
7. ✓ I understand that this lecture assumed three-phase balance and I know why L04 will revisit this.

References

- **Grainger & Stevenson,** *Power System Analysis*. The chapter on Newton-Raphson power flow.
- **Glover, Sarma, Overbye,** *Power System Analysis and Design*. Bus admittance matrix and per-unit basics.
- **Wood, Wollenberg, Sheble,** *Power Generation, Operation, and Control*. Decoupled and fast-decoupled PF.
- **pandapower documentation:** `https://pandapower.readthedocs.io` – "Balanced AC Power Flow".
- **Tinney & Hart, 1967.** *Power Flow Solution by Newton's Method*, IEEE TPAS. The original NR-PF paper, still readable.